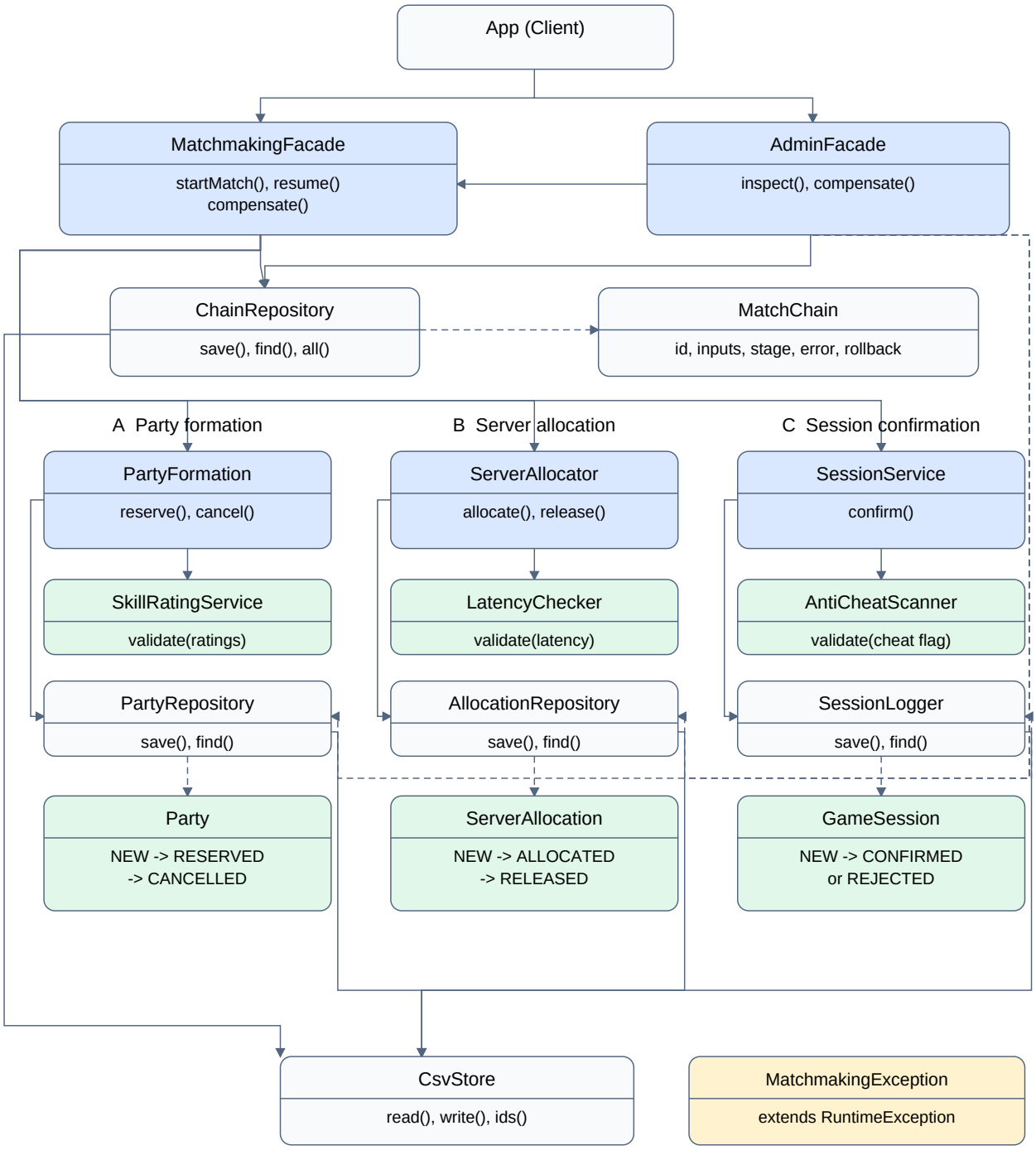
Software Design Patterns

Assignment 4, Video Game Matchmaking

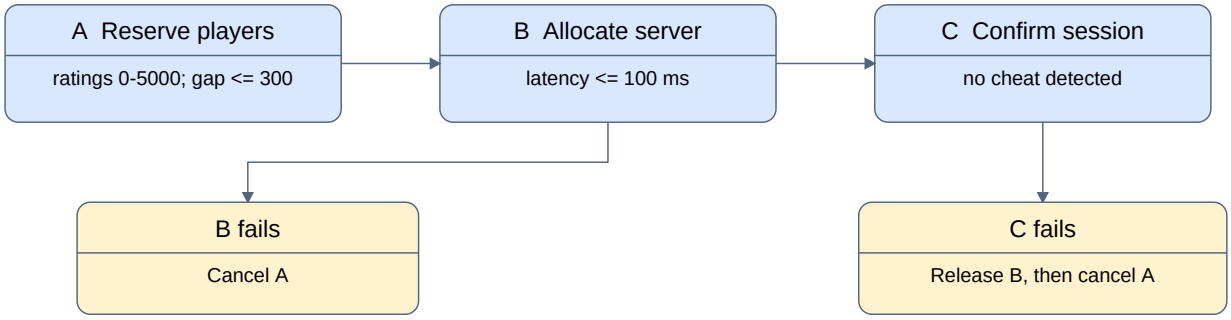
by Sabirzhanov Emil, SE-2533

Facade Class Diagram



Solid: stored reference. Dashed: reads or uses.

Matchmaking Workflow



Progress and entity statuses are saved in CSV files.

Design Rationale

The project matches two players for a game. A checks their ratings and reserves a party. B checks server latency and saves an allocation. C checks the cheat flag and confirms or rejects the session. Each subsystem has four classes: a service, a rule, an entity and a storage gateway.

The client calls MatchmakingFacade.startMatch(). The facade runs A, B and C in order. If B fails, it cancels A. If C fails, it releases B before cancelling A. AdminFacade shows saved records and cancels unfinished matches.

The chain saves its stage and inputs. After a restart, resume() reads the chain and subsystem records from disk. Saved steps are reused. If a write fails during rollback, the chain stays COMPENSATING so the rollback can be continued.

Facade fits because the client needs one operation over several subsystems. Adapter would be needed for an incompatible interface, such as a vendor server API. Here, the interfaces already work; the facade handles their order and failures.

There are 27 tests. Each uses a separate temporary folder, fixed inputs and JUnit assertions. They cover both rollback depths, restarts and failed writes. matchWith() is the domain test helper. Tests are committed with the code. JaCoCo reports 98.0% line coverage; the subsystems and facades have 100%.

This is a local demo. The user enters latency and the cheat flag; the application saves the result. It does not connect to game servers.

GitHub Repository

https://github.com/AlmazCode/SDP-A4